

Running GLM-5.2 (744B MoE) on a System76 Pangolin15 with colibrì

A weekend project disguised as a runbook. Or maybe a runbook disguised as a daisy-chained storage hierarchy.

If you have a System76 Pangolin15, Pop!_OS or Ubuntu, a fast local NVMe, and enough patience to let a 744B-parameter hummingbird sip from a fire hose, this guide gets you from “the model downloaded” to “colibrì is actually usable on my laptop” in about 30–60 minutes of setup plus however long your SSD takes to ingest 370 GB.[cite:2][web:7]

Final result on this class of machine:

- GLM-5.2 744B MoE int4 running locally through colibrì.[cite:2]
- Dense layers resident in RAM at about 9.9 GB, routed experts streamed from disk from a model that occupies about 370 GB on storage.[cite:2]
- A machine with 96 GB RAM can dedicate a large chunk of memory to hot-expert caching, which is where this stops feeling like a science fair volcano and starts feeling like a tool. [cite:2][web:7]

Class, gather round. Today’s lesson is about the oldest trick in computing history: if you cannot fit the whole cathedral into memory, you move only the stained-glass window you need right now. Colibrì does exactly that. GLM-5.2 is a 744B-parameter mixture-of-experts model, but only about 40B parameters are active per token, and only about 11 GB of those vary from token to token because the routed experts are streamed on demand from disk.[cite:2]

The dense part — embeddings, attention, shared experts, and the always-there architectural furniture — stays resident in RAM at int4 and consumes about 9.9 GB.[cite:2] The 19,456 routed experts live on disk at roughly 370 GB total and are pulled in as needed, with colibrì treating VRAM, RAM, and storage as one memory hierarchy instead of pretending your laptop is an H100 wearing a fake mustache.[cite:2]

TL;DR

The trick is simple enough to explain to a high school history class and weird enough to be funny. A 744B MoE model sounds impossible on a laptop, but the model only wakes a tiny fraction of its brain for each token, so colibrì keeps the always-needed parts in RAM and streams the specialized experts from NVMe when routing says they matter.[cite:2]

On a Pangolin15, the two big levers are RAM and disk. The pang15 supports up to 96 GB of DDR5 memory and PCIe Gen 4 NVMe storage, which makes it a much better fit for colibrì than the developer’s original 25 GB baseline box.[web:7][web:9][cite:2]

The history-teacher backstory

Colibri's core idea is not “make the model smaller.” It is “stop treating storage like a museum shelf and start treating it like the slowest level of memory.” The runtime is a tiny pure-C engine that preserves the model's precision and router behavior by default rather than quietly lobotomizing the checkpoint to make benchmarks look prettier.[cite:2]

That matters because the project's claim is unusually specific: GLM-5.2 744B MoE can run on a consumer machine with around 25 GB of RAM, no GPU required, by keeping the dense portion in RAM and streaming the experts from disk.[cite:2] It is not fast cold; the README says that plainly, with cold decode dominated by around 11 GB of disk reads per token and a baseline of roughly 0.05–0.1 tok/s on the developer's own 1 GB/s box.[cite:2]

Prerequisites

Hardware and software for the Pangolin15 version of this adventure:

- System76 Pangolin15, ideally the pang15 generation with up to 96 GB DDR5 and PCIe Gen 4 NVMe support.[web:7]
- Pop!_OS, Ubuntu, or another Linux environment with gcc and OpenMP available for the build.[cite:2]
- At least 16 GB RAM to run colibri at all, though 96 GB is where the hot-expert cache becomes genuinely useful.[cite:2][web:7]
- Roughly 400 GB free on a local NVMe path for the preconverted int4 model and some breathing room.[cite:2][web:19]
- Python only if downloading or converting model-side assets through Hugging Face tooling; the engine itself is pure C at runtime.[cite:2]

Verify the hardware before starting:

```
lscpu | grep "Model name"
lsblk -o NAME,SIZE,TYPE,MOUNTPOINT
free -h
```

What success looks like:

- CPU output naming your Ryzen processor.
- An NVMe volume with at least about 400 GB free.
- Enough RAM that reserving around 10–15 GB for the OS still leaves a large cache budget for experts.

Step 1 — Build the engine

From the repository root, go into the C runtime and let the project test itself:

```
cd colibri/c
```

```
./setup.sh
```

This script checks gcc and OpenMP, builds the engine, and runs self-tests against a tiny oracle model.[cite:2] The intended vibe is: before hauling 370 GB of expert shards onto your laptop, verify the bird cage is assembled correctly.

If `./setup.sh` finishes cleanly, the engine side is ready. If it fails, fix the compiler toolchain first; otherwise the later errors will be as educational as trying to teach Roman history with the wrong empire.

Step 2 — Put the model on fast local storage

The colibrì README recommends using the preconverted Hugging Face model with the fixed int8 MTP heads, specifically `mateogrgic/GLM-5.2-colibri-int4-with-int8-mtp`. [cite:2][web:19] That version matters because the original mirror with int4 MTP heads leaves speculative decoding stuck at effectively 0% draft acceptance, which means one of the project's main speed levers never engages. [cite:2][web:20]

Pick a directory you own on the NVMe, for example:

```
mkdir -p ~/models/glm52
```

Then, from the Python virtual environment used for Hugging Face tools:

```
source ~/colibri-venv/bin/activate
hf download mateogrgic/GLM-5.2-colibri-int4-with-int8-mtp \
  --local-dir ~/models/glm52 \
  --repo-type model
```

The Hugging Face docs support whole-repository downloads through `hf download`, which is the cleanest option for a giant model repository. [web:21][web:25] The model card for the `mateogrgic` repo also uses this exact pattern and calls for around 400 GB free on local NVMe. [web:19]

Step 3 — Verify you downloaded the right thing

After the download completes, check the directory and verify the three MTP files are the int8 versions:

```
ls -lh ~/models/glm52 | head
ls -l ~/models/glm52/out-mtp-*
```

Correct file sizes for the int8 MTP heads are listed in the README as 3527131672, 5366238584, and 1065950496 bytes. [cite:2] If you see the smaller int4 sizes instead, you downloaded the wrong mirror, and colibrì will behave like a student who showed up to the final exam with the wrong textbook. [cite:2]

Expected footprint:

Item	Expected value
Model on disk	~370 GB [cite:2]
Dense resident RAM	9.9 GB [cite:2]
Peak RSS during chat	~20 GB on the dev baseline; depends on cache budget [cite:2]
MTP heads	Int8-sized <code>out-mtp-*</code> files [cite:2]

Step 4 — Ask colibrì what it thinks of your machine

Before loading the model, use the planning and readiness commands:

```
COLI_MODEL=~/.models/glm52 ./coli doctor
COLI_MODEL=~/.models/glm52 ./coli plan
```

`coli doctor` performs a read-only readiness check over the model directory, safetensors headers, available RAM, and placement budget without starting inference.[cite:2] `coli plan` reads only headers as well and reports the exact dense and expert footprint, safe cache cap, and the planned storage hierarchy, which is much nicer than discovering a bad placement plan the exciting way.[cite:2]

If `doctor` passes and `plan` looks sane, move on. If not, fix the path, free space, or RAM assumptions before attempting chat.

Step 5 — First launch on a 96 GB Pangolin15

For a 96 GB system, the practical goal is to leave enough memory for the OS and let colibrì use the rest for expert caching. A conservative starting point is to claim about 82 GB for colibrì’s RAM budget while leaving headroom for the desktop, page cache, and ordinary civilization.[cite:2]

Start with:

```
COLI_MODEL=~/.models/glm52 ./coli chat --ram 82 --topp 0.7
```

The `--topp 0.7` knob is called out in the README and community results because it can cut disk traffic by roughly 30–40 percent by pruning colder experts during routing, which often buys a meaningful end-to-end speedup on disk-bound machines.[cite:2] This is not cheating so much as assigning fewer substitute teachers to the field trip.

Step 6 — Warm the cache like you mean it

Colibri includes a persistent learning cache. It records which experts your workload actually routes to in `.coli_usage` and can automatically pin the hottest ones in spare RAM on startup. [cite:2] This means the engine literally gets faster the more you use it, which is one of the few technology slogans that is both dramatic and true.[cite:2]

A simple warmup sequence:

```
# Build usage history
STATS=stats.txt COLI_MODEL=~/.models/glm52 ./coli chat

# Later, pin the hot set into RAM
PIN=stats.txt PIN_GB=65 COLI_MODEL=~/.models/glm52 ./coli chat --ram 82 --topp 0.7
```

The README describes this learned pinning flow and notes that a machine with more RAM benefits disproportionately because the hot set can live in memory instead of being replayed from disk over and over again.[cite:2] In plain English: stop making your SSD re-teach algebra every turn.

Step 7 — Measure your storage, because physics still has veto power

Colibri is honest about being disk-bound when cold. The project includes `iobench` specifically to measure the access pattern the engine uses: parallel reads over expert shards rather than fake-optimistic desktop copy speeds.[cite:2]

Build and run it like this:

```
gcc -O2 -fopenmp iobench.c -o iobench
./iobench ~/.models/glm52/out-00069.safetensors 19 64 8 1
```

The README's back-of-envelope guide says a native Linux machine with PCIe 4 NVMe at roughly 3–5 GB/s random and 32 GB RAM might see around 0.5–1 tok/s, while larger RAM budgets shift the bottleneck toward matmul instead of disk.[cite:2] With 96 GB RAM on a Pangolin15, the likely story is: painful when cold, noticeably better after the cache warms, and much more useful once hot experts stay resident.

What success looks like

A healthy startup looks something like this:

```
☒ colibri v1.0 — GLM-5.2 • 744B MoE • int4 • streaming CPU
✓ ready in ~30s • resident ~9.9 GB
```

Signs that the setup is wired correctly:

- `./setup.sh` passes.[cite:2]

- `hf download` populated the model directory with the safetensors and metadata expected by the repo.[web:19][web:21]
- `out-mtp-*` sizes match the int8 values from the README.[cite:2]
- `coli doctor` reports the configuration is runnable.[cite:2]
- `coli plan` shows a sane RAM cache budget and model placement.[cite:2]
- `coli chat --ram 82 --topp 0.7` starts without trying to swap the machine into the Stone Age.[cite:2]

Performance ceiling

The README’s “honest numbers” section is wonderfully blunt: this is not fast in the cold-start case, because a cold token can demand about 11 GB of disk reads across 75 layers and 8 experts per layer.[cite:2] On the developer’s own 25 GB WSL2 box with a roughly 1 GB/s drive, that translates to about 0.05–0.1 tok/s cold.[cite:2]

But the same document also shows why better machines help. More RAM increases expert hit rate, faster NVMe cuts cold miss pain, and MTP speculation with the correct int8 heads can reach 2.2–2.8 tokens per forward when the cache is warm enough that the extra routing work is worth it.[cite:2]

A practical expectation table for a Pangolin15-class machine:

Condition	Expected feel
First cold run	Slow enough to make you contemplate monastery life [cite:2]
After several chats	Better, because <code>.coli_usage</code> and the OS page cache start helping [cite:2]
Warm cache + hot expert pin	Genuinely useful for local reasoning and coding, depending on SSD speed [cite:2][web:7]
Wrong MTP-head mirror	Speculation stays stuck and one major speed lever disappears [cite:2][web:20]

Useful follow-on tools

OpenAI-compatible local API:

```
COLI_MODEL=~/.models/glm52 COLI_API_KEY=local-secret ./coli serve \
--host 127.0.0.1 --port 8000 --model-id glm-5.2-colibri
```

The README documents `coli serve` as a text-only OpenAI-compatible HTTP server with chat completions, SSE streaming, queueing, and explicit limits around unsupported features such as tool use and multimodal input.[cite:2] It keeps one model process loaded, which is exactly what you want when the model is large enough to make repeated loads feel like moving a piano upstairs alone.[cite:2]

Web interface:

```
cd web
```

```
npm ci && npm run dev
```

The repository includes a community-maintained browser UI that talks to any OpenAI-compatible endpoint and never touches the engine directly.[cite:2] The main dashboard path in the README is `./coli web`, which can serve both the API and the console on one port once the web assets are built.[cite:2]

Troubleshooting

Health check

Run these first when something breaks:

```
# 1. Engine still builds?
cd ~/colibri/c && ./setup.sh

# 2. Model path still valid?
ls ~/models/glm52/out-mtp-*

# 3. Readiness still passes?
COLI_MODEL=~/.models/glm52 ./coli doctor

# 4. Placement still sane?
COLI_MODEL=~/.models/glm52 ./coli plan
```

If all four pass and the system still feels wrong, the most likely culprits are a changed RAM budget, a slow or nearly full SSD, or accidentally using the wrong model mirror.[cite:2][web:19][web:20]

MTP stuck at 0%

Cause: wrong mirror or wrong `out-mtp-*` files. The README says this is the single most common reason MTP never engages, because the original mirror ships int4 MTP heads instead of int8.[cite:2]

Fix:

- Check `ls -l ~/.models/glm52/out-mtp-*`.
- Compare sizes against the README's int8 values.[cite:2]
- Replace just those three files from the `mateogrgic` mirror if needed.[cite:2][web:19]

Model is unbearably slow

This is often not “broken” so much as “cold.” The README notes that cold decode is dominated by disk traffic and that cache hit rate is the whole game on modest hardware.[cite:2]

Try these in order:

- Use `--topp 0.7`.[cite:2]

- Give colibrì an explicit RAM budget, such as `--ram 82` on a 96 GB machine.[cite:2]
- Build usage history and pin the hot experts with `PIN_GB`. [cite:2]
- Run `iobench` to find out whether your SSD is the real villain instead of blaming the bird. [cite:2]

Python install headaches on Ubuntu or Pop!_OS

If `pip install huggingface_hub` throws `externally-managed-environment`, that is the distro protecting the system Python. The recommended workaround is to use a virtual environment with `python3 -m venv` and install the Hugging Face tools inside it rather than forcing packages into the OS-managed interpreter.[web:34][web:47][web:48]

Closing thought

You are not running “a small local model.” You are running a frontier-class 744B mixture-of-experts model on a consumer laptop by making storage, RAM, and software discipline cooperate like a well-rehearsed stage cast. That is less like buying a sports car and more like teaching a hummingbird to tow a cathedral.[cite:2]

The Pangolin15 is a particularly good stage for this trick because its high RAM ceiling and Gen 4 NVMe support align with exactly the two resources colibrì exploits best.[web:7][web:9][cite:2] The result is not magic, but it is the kind of engineering magic trick that makes history teachers suspiciously fond of Linux.